Page 1: Handoff Objective and Current Status

Page purpose: This manual exists so another Codex instance can start product work without needing the whole chat. The current build is a Windows Node host plus phone PWA with local Wi-Fi control, host approval, real input, WebRTC high-quality stream, packaged zip, and a known v76 fix for touchpad/zoom behavior.

- Treat `C:\RemoteDesktopControllerPlan\remote-control-mvp` as the source workspace.
- The project must be moved to the user's computer, then into a private GitHub repository before broad parallel work.
- Known unresolved issue: multi-monitor switching can still drift or return to the wrong screen; do not hide this from the next worker.

Exact operator actions

For 'Handoff Objective and Current Status', the operator should follow these steps rather than improvising from memory.

- Read the whole page once before touching files, Git, GitHub, package artifacts, or another computer.
- Capture current evidence first: folder path, branch or no-branch state, package version, latest zip path, and the commands that currently pass.
- Perform the named transfer or handoff action in one small batch, then verify immediately before moving to the next batch.
- Record any deviation in the handoff notes so the next Codex instance does not assume the ideal path happened.

Step-by-step handoff script

This is the minimum practical script for 'Handoff Objective and Current Status'. The point is to make the transfer repeatable for a new machine or a new Codex instance, not just understandable to the person who already lived through the prototype.

- Step 1: confirm the current workspace path and latest package path, then write both paths in the issue or handoff note before copying or editing anything.
- Step 2: run the listed smoke checks from the current machine so the sender knows whether the outgoing build is already healthy or already broken.
- Step 3: copy or publish only the approved source/package set. Exclude local runtime data, logs, PID files, extracted smoke folders, `node_modules`, and disabled scratch files.
- Step 4: on the receiving machine, launch from the documented command or the single packaged launcher and verify that the host page, phone URL, QR code, PIN, pairing, and real-input toggle appear as expected.
- Step 5: run the core automated tests from the receiving source checkout, or if using only the family package, run the package smoke and one phone pairing test.
- Step 6: record the result with machine name, Windows version, browser version, phone type, network type, and any missing permissions or firewall prompts.

What must be true before the next person starts

A new worker should not begin creative implementation until the handoff basics are proven. This prevents debugging a broken transfer while pretending to build product features.

- The project opens from the documented folder and the expected files are present with normal names.
- The host can start or the package launcher can start without missing Node, missing modules, blocked scripts, or stale disabled files.
- The phone-facing page can be reached through the documented local IP or tunnel path and shows the current UI rather than an old cached version.
- The worker knows whether they are editing source, testing a package, creating a GitHub issue, or preparing a release artifact.
- The worker knows which known issues are accepted for now, especially monitor switching, native capture, remote relay economics, and app-store constraints.

Files and state to protect

These are the project assets that should not be casually deleted, renamed, or overwritten during the transfer.

- `src/server.js`, `src/input-adapter.js`, `src/rtc-room.js`, `src/session-store.js`, `public/app.js`, `public/capture.js`, `public/host.js`, `public/sw.js`, and `packaging/local-wifi/*`.
- `docs/product_planning/*`, `README.md`, `MILESTONE_STATUS.md`, existing traceability docs, and checkpoint artifacts under `output/checkpoints`.
- `dist/OpenHostLink.zip` as the latest family-testable package, while treating it as a release artifact rather than source.
- Local secrets and runtime data under `data`, logs, PID files, and smoke-test extraction folders must be excluded from GitHub unless sanitized.

Verification before handoff is accepted

A handoff is not complete because files were copied. It is complete when the recipient can run and verify the system.

- The recipient can install dependencies or use the packaged zip without missing-file errors.
- The host console opens, shows the correct version, and exposes a phone URL/QR code.
- The test command listed in this manual passes or has an explicit, documented reason why a physical-device gate is still pending.
- The next Codex instance has a branch, issue, file ownership boundary, and acceptance gate before it starts editing.

Common mistakes to avoid

The transfer should avoid mistakes that already caused confusion during the prototype phase.

- Do not ship a package with multiple confusing launchers when one obvious launcher is expected.
- Do not let `.disabled-by-incident-response` file names become the canonical GitHub source state.
- Do not describe the monitor-switching system as fixed until it passes multi-monitor testing on another computer.
- Do not let a paid remote-access experiment break the free local Wi-Fi mode, because the free mode is the project's credibility base.

Acceptance evidence to attach

Every handoff page should produce evidence. If the page does not create evidence, it is probably only a note and not a real manual step.

- Attach command output or a short transcript for every command that matters.
- Attach a screenshot when the result is visual, such as host console, QR code, package folder, phone UI, capture window, settings sheet, or display selector.
- Attach file hashes for release zips so the sender and receiver can prove they are testing the same artifact.
- Attach a short failed-test note when something is known broken. A truthful failed-test note is more useful than a vague 'needs polish' sentence.
- Attach the next action owner so the handoff does not end in a pile of unassigned concerns.

Completion goals: a new worker can understand the page without chat history; the page gives concrete commands, files, or rules; and every named risk maps to the 25-page plan.

Page 2: Core Architecture

Page purpose: The host serves the phone PWA, host console, capture page, REST APIs, WebSocket control channel, and RTC signaling. The phone sends validated commands; the host maps them to Windows input through the input adapter after safety approval.

- `src/server.js` owns HTTP, WebSocket, capture launch, host state, safety endpoints, and RTC room integration.
- `src/input-adapter.js` owns real/dry-run mouse, wheel, keyboard, chord, and text input.
- `public/app.js` owns phone UI, pairing, touchpad, zoom, keyboard, monitor selection, RTC receiver, and stream rendering.
- `public/capture.js` owns the low-latency capture browser path.
- `packaging/local-wifi` builds the easy-send zip.

Exact operator actions

For 'Core Architecture', the operator should follow these steps rather than improvising from memory.

- Read the whole page once before touching files, Git, GitHub, package artifacts, or another computer.
- Capture current evidence first: folder path, branch or no-branch state, package version, latest zip path, and the commands that currently pass.
- Perform the named transfer or handoff action in one small batch, then verify immediately before moving to the next batch.
- Record any deviation in the handoff notes so the next Codex instance does not assume the ideal path happened.

Step-by-step handoff script

This is the minimum practical script for 'Core Architecture'. The point is to make the transfer repeatable for a new machine or a new Codex instance, not just understandable to the person who already lived through the prototype.

- Step 1: confirm the current workspace path and latest package path, then write both paths in the issue or handoff note before copying or editing anything.
- Step 2: run the listed smoke checks from the current machine so the sender knows whether the outgoing build is already healthy or already broken.
- Step 3: copy or publish only the approved source/package set. Exclude local runtime data, logs, PID files, extracted smoke folders, `node_modules`, and disabled scratch files.
- Step 4: on the receiving machine, launch from the documented command or the single packaged launcher and verify that the host page, phone URL, QR code, PIN, pairing, and real-input toggle appear as expected.
- Step 5: run the core automated tests from the receiving source checkout, or if using only the family package, run the package smoke and one phone pairing test.
- Step 6: record the result with machine name, Windows version, browser version, phone type, network type, and any missing permissions or firewall prompts.

What must be true before the next person starts

A new worker should not begin creative implementation until the handoff basics are proven. This prevents debugging a broken transfer while pretending to build product features.

- The project opens from the documented folder and the expected files are present with normal names.
- The host can start or the package launcher can start without missing Node, missing modules, blocked scripts, or stale disabled files.
- The phone-facing page can be reached through the documented local IP or tunnel path and shows the current UI rather than an old cached version.
- The worker knows whether they are editing source, testing a package, creating a GitHub issue, or preparing a release artifact.
- The worker knows which known issues are accepted for now, especially monitor switching, native capture, remote relay economics, and app-store constraints.

Files and state to protect

These are the project assets that should not be casually deleted, renamed, or overwritten during the transfer.

- `src/server.js`, `src/input-adapter.js`, `src/rtc-room.js`, `src/session-store.js`, `public/app.js`, `public/capture.js`, `public/host.js`, `public/sw.js`, and `packaging/local-wifi/*`.
- `docs/product_planning/*`, `README.md`, `MILESTONE_STATUS.md`, existing traceability docs, and checkpoint artifacts under `output/checkpoints`.
- `dist/OpenHostLink.zip` as the latest family-testable package, while treating it as a release artifact rather than source.
- Local secrets and runtime data under `data`, logs, PID files, and smoke-test extraction folders must be excluded from GitHub unless sanitized.

Verification before handoff is accepted

A handoff is not complete because files were copied. It is complete when the recipient can run and verify the system.

- The recipient can install dependencies or use the packaged zip without missing-file errors.
- The host console opens, shows the correct version, and exposes a phone URL/QR code.
- The test command listed in this manual passes or has an explicit, documented reason why a physical-device gate is still pending.
- The next Codex instance has a branch, issue, file ownership boundary, and acceptance gate before it starts editing.

Common mistakes to avoid

The transfer should avoid mistakes that already caused confusion during the prototype phase.

- Do not ship a package with multiple confusing launchers when one obvious launcher is expected.
- Do not let `.disabled-by-incident-response` file names become the canonical GitHub source state.
- Do not describe the monitor-switching system as fixed until it passes multi-monitor testing on another computer.
- Do not let a paid remote-access experiment break the free local Wi-Fi mode, because the free mode is the project's credibility base.

Acceptance evidence to attach

Every handoff page should produce evidence. If the page does not create evidence, it is probably only a note and not a real manual step.

- Attach command output or a short transcript for every command that matters.
- Attach a screenshot when the result is visual, such as host console, QR code, package folder, phone UI, capture window, settings sheet, or display selector.
- Attach file hashes for release zips so the sender and receiver can prove they are testing the same artifact.
- Attach a short failed-test note when something is known broken. A truthful failed-test note is more useful than a vague 'needs polish' sentence.
- Attach the next action owner so the handoff does not end in a pile of unassigned concerns.

Completion goals: a new worker can understand the page without chat history; the page gives concrete commands, files, or rules; and every named risk maps to the 25-page plan.

Page 3: Run, Test, and Package Commands

Page purpose: The baseline commands are intentionally simple. A new worker should run them before editing and after any meaningful change.

- Run host: `$env:HOST_KEY='dev-host-key'; npm start`.
- Open host: `http://127.0.0.1:4317/host?key=dev-host-key`.
- Run core tests: `node --test test\protocol.test.js test\rtc-room.test.js test\settings-store.test.js test\session-store.test.js test\coordinate-mapper.test.js test\input-adapter.test.js test\host-console.test.js test\phone-connection-help.test.js`.
- Package: `powershell -NoProfile -ExecutionPolicy Bypass -File packaging\local-wifi\package-local-wifi.ps1 -PackageName OpenHostLink`.
- Smoke package on a free port before sending it to family testers.

Exact operator actions

For 'Run, Test, and Package Commands', the operator should follow these steps rather than improvising from memory.

- Read the whole page once before touching files, Git, GitHub, package artifacts, or another computer.
- Capture current evidence first: folder path, branch or no-branch state, package version, latest zip path, and the commands that currently pass.
- Perform the named transfer or handoff action in one small batch, then verify immediately before moving to the next batch.
- Record any deviation in the handoff notes so the next Codex instance does not assume the ideal path happened.

Step-by-step handoff script

This is the minimum practical script for 'Run, Test, and Package Commands'. The point is to make the transfer repeatable for a new machine or a new Codex instance, not just understandable to the person who already lived through the prototype.

- Step 1: confirm the current workspace path and latest package path, then write both paths in the issue or handoff note before copying or editing anything.
- Step 2: run the listed smoke checks from the current machine so the sender knows whether the outgoing build is already healthy or already broken.
- Step 3: copy or publish only the approved source/package set. Exclude local runtime data, logs, PID files, extracted smoke folders, `node_modules`, and disabled scratch files.
- Step 4: on the receiving machine, launch from the documented command or the single packaged launcher and verify that the host page, phone URL, QR code, PIN, pairing, and real-input toggle appear as expected.
- Step 5: run the core automated tests from the receiving source checkout, or if using only the family package, run the package smoke and one phone pairing test.
- Step 6: record the result with machine name, Windows version, browser version, phone type, network type, and any missing permissions or firewall prompts.

What must be true before the next person starts

A new worker should not begin creative implementation until the handoff basics are proven. This prevents debugging a broken transfer while pretending to build product features.

- The project opens from the documented folder and the expected files are present with normal names.
- The host can start or the package launcher can start without missing Node, missing modules, blocked scripts, or stale disabled files.
- The phone-facing page can be reached through the documented local IP or tunnel path and shows the current UI rather than an old cached version.
- The worker knows whether they are editing source, testing a package, creating a GitHub issue, or preparing a release artifact.
- The worker knows which known issues are accepted for now, especially monitor switching, native capture, remote relay economics, and app-store constraints.

Files and state to protect

These are the project assets that should not be casually deleted, renamed, or overwritten during the transfer.

- `src/server.js`, `src/input-adapter.js`, `src/rtc-room.js`, `src/session-store.js`, `public/app.js`, `public/capture.js`, `public/host.js`, `public/sw.js`, and `packaging/local-wifi/*`.
- `docs/product_planning/*`, `README.md`, `MILESTONE_STATUS.md`, existing traceability docs, and checkpoint artifacts under `output/checkpoints`.
- `dist/OpenHostLink.zip` as the latest family-testable package, while treating it as a release artifact rather than source.
- Local secrets and runtime data under `data`, logs, PID files, and smoke-test extraction folders must be excluded from GitHub unless sanitized.

Verification before handoff is accepted

A handoff is not complete because files were copied. It is complete when the recipient can run and verify the system.

- The recipient can install dependencies or use the packaged zip without missing-file errors.
- The host console opens, shows the correct version, and exposes a phone URL/QR code.
- The test command listed in this manual passes or has an explicit, documented reason why a physical-device gate is still pending.
- The next Codex instance has a branch, issue, file ownership boundary, and acceptance gate before it starts editing.

Common mistakes to avoid

The transfer should avoid mistakes that already caused confusion during the prototype phase.

- Do not ship a package with multiple confusing launchers when one obvious launcher is expected.
- Do not let `.disabled-by-incident-response` file names become the canonical GitHub source state.
- Do not describe the monitor-switching system as fixed until it passes multi-monitor testing on another computer.
- Do not let a paid remote-access experiment break the free local Wi-Fi mode, because the free mode is the project's credibility base.

Acceptance evidence to attach

Every handoff page should produce evidence. If the page does not create evidence, it is probably only a note and not a real manual step.

- Attach command output or a short transcript for every command that matters.
- Attach a screenshot when the result is visual, such as host console, QR code, package folder, phone UI, capture window, settings sheet, or display selector.
- Attach file hashes for release zips so the sender and receiver can prove they are testing the same artifact.
- Attach a short failed-test note when something is known broken. A truthful failed-test note is more useful than a vague 'needs polish' sentence.
- Attach the next action owner so the handoff does not end in a pile of unassigned concerns.

Completion goals: a new worker can understand the page without chat history; the page gives concrete commands, files, or rules; and every named risk maps to the 25-page plan.

Page 4: Transfer to User Computer

Page purpose: The safest transfer is a GitHub repository plus release zip. If GitHub is not ready yet, transfer the whole folder excluding generated junk and verify on the user's PC.

- Copy source folders: `src`, `public`, `packaging`, `scripts`, `test`, `docs`, root `README.md`, `package.json`, `package-lock.json`, and current `dist\OpenHostLink.zip`.
- Exclude `node_modules`, most `output` smoke folders, logs, PID files, local data/trust stores, and any personal screenshots.
- On the new PC install Node 20+, run `npm ci`, then run the tests and package smoke.
- If running from zip, use `Start Remote Controller.bat` or `start-local-wifi.ps1`; if running source, use `npm start`.

Exact operator actions

For 'Transfer to User Computer', the operator should follow these steps rather than improvising from memory.

- Read the whole page once before touching files, Git, GitHub, package artifacts, or another computer.
- Capture current evidence first: folder path, branch or no-branch state, package version, latest zip path, and the commands that currently pass.
- Perform the named transfer or handoff action in one small batch, then verify immediately before moving to the next batch.
- Record any deviation in the handoff notes so the next Codex instance does not assume the ideal path happened.

Step-by-step handoff script

This is the minimum practical script for 'Transfer to User Computer'. The point is to make the transfer repeatable for a new machine or a new Codex instance, not just understandable to the person who already lived through the prototype.

- Step 1: confirm the current workspace path and latest package path, then write both paths in the issue or handoff note before copying or editing anything.
- Step 2: run the listed smoke checks from the current machine so the sender knows whether the outgoing build is already healthy or already broken.
- Step 3: copy or publish only the approved source/package set. Exclude local runtime data, logs, PID files, extracted smoke folders, `node_modules`, and disabled scratch files.
- Step 4: on the receiving machine, launch from the documented command or the single packaged launcher and verify that the host page, phone URL, QR code, PIN, pairing, and real-input toggle appear as expected.
- Step 5: run the core automated tests from the receiving source checkout, or if using only the family package, run the package smoke and one phone pairing test.
- Step 6: record the result with machine name, Windows version, browser version, phone type, network type, and any missing permissions or firewall prompts.

What must be true before the next person starts

A new worker should not begin creative implementation until the handoff basics are proven. This prevents debugging a broken transfer while pretending to build product features.

- The project opens from the documented folder and the expected files are present with normal names.
- The host can start or the package launcher can start without missing Node, missing modules, blocked scripts, or stale disabled files.
- The phone-facing page can be reached through the documented local IP or tunnel path and shows the current UI rather than an old cached version.
- The worker knows whether they are editing source, testing a package, creating a GitHub issue, or preparing a release artifact.
- The worker knows which known issues are accepted for now, especially monitor switching, native capture, remote relay economics, and app-store constraints.

Files and state to protect

These are the project assets that should not be casually deleted, renamed, or overwritten during the transfer.

- `src/server.js`, `src/input-adapter.js`, `src/rtc-room.js`, `src/session-store.js`, `public/app.js`, `public/capture.js`, `public/host.js`, `public/sw.js`, and `packaging/local-wifi/*`.
- `docs/product_planning/*`, `README.md`, `MILESTONE_STATUS.md`, existing traceability docs, and checkpoint artifacts under `output/checkpoints`.
- `dist/OpenHostLink.zip` as the latest family-testable package, while treating it as a release artifact rather than source.
- Local secrets and runtime data under `data`, logs, PID files, and smoke-test extraction folders must be excluded from GitHub unless sanitized.

Verification before handoff is accepted

A handoff is not complete because files were copied. It is complete when the recipient can run and verify the system.

- The recipient can install dependencies or use the packaged zip without missing-file errors.
- The host console opens, shows the correct version, and exposes a phone URL/QR code.
- The test command listed in this manual passes or has an explicit, documented reason why a physical-device gate is still pending.
- The next Codex instance has a branch, issue, file ownership boundary, and acceptance gate before it starts editing.

Common mistakes to avoid

The transfer should avoid mistakes that already caused confusion during the prototype phase.

- Do not ship a package with multiple confusing launchers when one obvious launcher is expected.
- Do not let `.disabled-by-incident-response` file names become the canonical GitHub source state.
- Do not describe the monitor-switching system as fixed until it passes multi-monitor testing on another computer.
- Do not let a paid remote-access experiment break the free local Wi-Fi mode, because the free mode is the project's credibility base.

Acceptance evidence to attach

Every handoff page should produce evidence. If the page does not create evidence, it is probably only a note and not a real manual step.

- Attach command output or a short transcript for every command that matters.
- Attach a screenshot when the result is visual, such as host console, QR code, package folder, phone UI, capture window, settings sheet, or display selector.
- Attach file hashes for release zips so the sender and receiver can prove they are testing the same artifact.
- Attach a short failed-test note when something is known broken. A truthful failed-test note is more useful than a vague 'needs polish' sentence.
- Attach the next action owner so the handoff does not end in a pile of unassigned concerns.

Completion goals: a new worker can understand the page without chat history; the page gives concrete commands, files, or rules; and every named risk maps to the 25-page plan.

Page 5: GitHub Setup Plan

Page purpose: Create a private repository first. Public/open-source release can come after security, licensing, and packaging are cleaner.

- Initialize Git in the project root only after `.gitignore` is created.
- First commit: source, docs, tests, packaging scripts, package manifests; no `node_modules`, logs, host keys, or generated smoke output.
- Tag the imported baseline as `v0.1-local-wifi-baseline-v76`.
- Create Issues from the 25-page plan; each issue names owner, branch, files, tests, acceptance evidence.
- Use pull requests for every Codex instance branch.

Exact operator actions

For 'GitHub Setup Plan', the operator should follow these steps rather than improvising from memory.

- Read the whole page once before touching files, Git, GitHub, package artifacts, or another computer.
- Capture current evidence first: folder path, branch or no-branch state, package version, latest zip path, and the commands that currently pass.
- Perform the named transfer or handoff action in one small batch, then verify immediately before moving to the next batch.
- Record any deviation in the handoff notes so the next Codex instance does not assume the ideal path happened.

Step-by-step handoff script

This is the minimum practical script for 'GitHub Setup Plan'. The point is to make the transfer repeatable for a new machine or a new Codex instance, not just understandable to the person who already lived through the prototype.

- Step 1: confirm the current workspace path and latest package path, then write both paths in the issue or handoff note before copying or editing anything.
- Step 2: run the listed smoke checks from the current machine so the sender knows whether the outgoing build is already healthy or already broken.
- Step 3: copy or publish only the approved source/package set. Exclude local runtime data, logs, PID files, extracted smoke folders, `node_modules`, and disabled scratch files.
- Step 4: on the receiving machine, launch from the documented command or the single packaged launcher and verify that the host page, phone URL, QR code, PIN, pairing, and real-input toggle appear as expected.
- Step 5: run the core automated tests from the receiving source checkout, or if using only the family package, run the package smoke and one phone pairing test.
- Step 6: record the result with machine name, Windows version, browser version, phone type, network type, and any missing permissions or firewall prompts.

What must be true before the next person starts

A new worker should not begin creative implementation until the handoff basics are proven. This prevents debugging a broken transfer while pretending to build product features.

- The project opens from the documented folder and the expected files are present with normal names.
- The host can start or the package launcher can start without missing Node, missing modules, blocked scripts, or stale disabled files.
- The phone-facing page can be reached through the documented local IP or tunnel path and shows the current UI rather than an old cached version.
- The worker knows whether they are editing source, testing a package, creating a GitHub issue, or preparing a release artifact.
- The worker knows which known issues are accepted for now, especially monitor switching, native capture, remote relay economics, and app-store constraints.

Files and state to protect

These are the project assets that should not be casually deleted, renamed, or overwritten during the transfer.

- `src/server.js`, `src/input-adapter.js`, `src/rtc-room.js`, `src/session-store.js`, `public/app.js`, `public/capture.js`, `public/host.js`, `public/sw.js`, and `packaging/local-wifi/*`.
- `docs/product_planning/*`, `README.md`, `MILESTONE_STATUS.md`, existing traceability docs, and checkpoint artifacts under `output/checkpoints`.
- `dist/OpenHostLink.zip` as the latest family-testable package, while treating it as a release artifact rather than source.
- Local secrets and runtime data under `data`, logs, PID files, and smoke-test extraction folders must be excluded from GitHub unless sanitized.

Verification before handoff is accepted

A handoff is not complete because files were copied. It is complete when the recipient can run and verify the system.

- The recipient can install dependencies or use the packaged zip without missing-file errors.
- The host console opens, shows the correct version, and exposes a phone URL/QR code.
- The test command listed in this manual passes or has an explicit, documented reason why a physical-device gate is still pending.
- The next Codex instance has a branch, issue, file ownership boundary, and acceptance gate before it starts editing.

Common mistakes to avoid

The transfer should avoid mistakes that already caused confusion during the prototype phase.

- Do not ship a package with multiple confusing launchers when one obvious launcher is expected.
- Do not let `.disabled-by-incident-response` file names become the canonical GitHub source state.
- Do not describe the monitor-switching system as fixed until it passes multi-monitor testing on another computer.
- Do not let a paid remote-access experiment break the free local Wi-Fi mode, because the free mode is the project's credibility base.

Acceptance evidence to attach

Every handoff page should produce evidence. If the page does not create evidence, it is probably only a note and not a real manual step.

- Attach command output or a short transcript for every command that matters.
- Attach a screenshot when the result is visual, such as host console, QR code, package folder, phone UI, capture window, settings sheet, or display selector.
- Attach file hashes for release zips so the sender and receiver can prove they are testing the same artifact.
- Attach a short failed-test note when something is known broken. A truthful failed-test note is more useful than a vague 'needs polish' sentence.
- Attach the next action owner so the handoff does not end in a pile of unassigned concerns.

Completion goals: a new worker can understand the page without chat history; the page gives concrete commands, files, or rules; and every named risk maps to the 25-page plan.

Page 6: Parallel Codex Rules

Page purpose: Every Codex instance needs a narrow workstream. Broad instructions like 'make it better' cause collisions.

- Assign one chapter, one branch, and one file ownership boundary.
- Do not change shared protocol/auth/capture metadata without an interface note.
- Do not remove tests to pass CI.
- Do not rewrite the phone UI and host protocol in the same branch unless the issue explicitly owns both.
- Final report must include changed files, tests, screenshots/artifacts, risks, and next issue.

Exact operator actions

For 'Parallel Codex Rules', the operator should follow these steps rather than improvising from memory.

- Read the whole page once before touching files, Git, GitHub, package artifacts, or another computer.
- Capture current evidence first: folder path, branch or no-branch state, package version, latest zip path, and the commands that currently pass.
- Perform the named transfer or handoff action in one small batch, then verify immediately before moving to the next batch.
- Record any deviation in the handoff notes so the next Codex instance does not assume the ideal path happened.

Step-by-step handoff script

This is the minimum practical script for 'Parallel Codex Rules'. The point is to make the transfer repeatable for a new machine or a new Codex instance, not just understandable to the person who already lived through the prototype.

- Step 1: confirm the current workspace path and latest package path, then write both paths in the issue or handoff note before copying or editing anything.
- Step 2: run the listed smoke checks from the current machine so the sender knows whether the outgoing build is already healthy or already broken.
- Step 3: copy or publish only the approved source/package set. Exclude local runtime data, logs, PID files, extracted smoke folders, `node_modules`, and disabled scratch files.
- Step 4: on the receiving machine, launch from the documented command or the single packaged launcher and verify that the host page, phone URL, QR code, PIN, pairing, and real-input toggle appear as expected.
- Step 5: run the core automated tests from the receiving source checkout, or if using only the family package, run the package smoke and one phone pairing test.
- Step 6: record the result with machine name, Windows version, browser version, phone type, network type, and any missing permissions or firewall prompts.

What must be true before the next person starts

A new worker should not begin creative implementation until the handoff basics are proven. This prevents debugging a broken transfer while pretending to build product features.

- The project opens from the documented folder and the expected files are present with normal names.
- The host can start or the package launcher can start without missing Node, missing modules, blocked scripts, or stale disabled files.
- The phone-facing page can be reached through the documented local IP or tunnel path and shows the current UI rather than an old cached version.
- The worker knows whether they are editing source, testing a package, creating a GitHub issue, or preparing a release artifact.
- The worker knows which known issues are accepted for now, especially monitor switching, native capture, remote relay economics, and app-store constraints.

Files and state to protect

These are the project assets that should not be casually deleted, renamed, or overwritten during the transfer.

- `src/server.js`, `src/input-adapter.js`, `src/rtc-room.js`, `src/session-store.js`, `public/app.js`, `public/capture.js`, `public/host.js`, `public/sw.js`, and `packaging/local-wifi/*`.
- `docs/product_planning/*`, `README.md`, `MILESTONE_STATUS.md`, existing traceability docs, and checkpoint artifacts under `output/checkpoints`.
- `dist/OpenHostLink.zip` as the latest family-testable package, while treating it as a release artifact rather than source.
- Local secrets and runtime data under `data`, logs, PID files, and smoke-test extraction folders must be excluded from GitHub unless sanitized.

Verification before handoff is accepted

A handoff is not complete because files were copied. It is complete when the recipient can run and verify the system.

- The recipient can install dependencies or use the packaged zip without missing-file errors.
- The host console opens, shows the correct version, and exposes a phone URL/QR code.
- The test command listed in this manual passes or has an explicit, documented reason why a physical-device gate is still pending.
- The next Codex instance has a branch, issue, file ownership boundary, and acceptance gate before it starts editing.

Common mistakes to avoid

The transfer should avoid mistakes that already caused confusion during the prototype phase.

- Do not ship a package with multiple confusing launchers when one obvious launcher is expected.
- Do not let `.disabled-by-incident-response` file names become the canonical GitHub source state.
- Do not describe the monitor-switching system as fixed until it passes multi-monitor testing on another computer.
- Do not let a paid remote-access experiment break the free local Wi-Fi mode, because the free mode is the project's credibility base.

Acceptance evidence to attach

Every handoff page should produce evidence. If the page does not create evidence, it is probably only a note and not a real manual step.

- Attach command output or a short transcript for every command that matters.
- Attach a screenshot when the result is visual, such as host console, QR code, package folder, phone UI, capture window, settings sheet, or display selector.
- Attach file hashes for release zips so the sender and receiver can prove they are testing the same artifact.
- Attach a short failed-test note when something is known broken. A truthful failed-test note is more useful than a vague 'needs polish' sentence.
- Attach the next action owner so the handoff does not end in a pile of unassigned concerns.

Completion goals: a new worker can understand the page without chat history; the page gives concrete commands, files, or rules; and every named risk maps to the 25-page plan.

Page 7: Known Issues and Product Risks

Page purpose: Known issues are part of the handoff, not embarrassment. They prevent repeated blind fixes.

- Multi-monitor switching remains the biggest current functional risk.
- Browser capture cannot be the final unattended paid remote strategy because screen capture permission is user-mediated [S1].
- The project currently relies on Chrome/browser capture for high-quality stream; native capture/encode is the likely paid-product path.
- Relay bandwidth can destroy subscription margin; Cloudflare is promising for early TURN but must be measured [S4].
- App Store review needs careful positioning as a generic remote desktop client [S7].

Exact operator actions

For 'Known Issues and Product Risks', the operator should follow these steps rather than improvising from memory.

- Read the whole page once before touching files, Git, GitHub, package artifacts, or another computer.
- Capture current evidence first: folder path, branch or no-branch state, package version, latest zip path, and the commands that currently pass.
- Perform the named transfer or handoff action in one small batch, then verify immediately before moving to the next batch.
- Record any deviation in the handoff notes so the next Codex instance does not assume the ideal path happened.

Step-by-step handoff script

This is the minimum practical script for 'Known Issues and Product Risks'. The point is to make the transfer repeatable for a new machine or a new Codex instance, not just understandable to the person who already lived through the prototype.

- Step 1: confirm the current workspace path and latest package path, then write both paths in the issue or handoff note before copying or editing anything.
- Step 2: run the listed smoke checks from the current machine so the sender knows whether the outgoing build is already healthy or already broken.
- Step 3: copy or publish only the approved source/package set. Exclude local runtime data, logs, PID files, extracted smoke folders, `node_modules`, and disabled scratch files.
- Step 4: on the receiving machine, launch from the documented command or the single packaged launcher and verify that the host page, phone URL, QR code, PIN, pairing, and real-input toggle appear as expected.
- Step 5: run the core automated tests from the receiving source checkout, or if using only the family package, run the package smoke and one phone pairing test.
- Step 6: record the result with machine name, Windows version, browser version, phone type, network type, and any missing permissions or firewall prompts.

What must be true before the next person starts

A new worker should not begin creative implementation until the handoff basics are proven. This prevents debugging a broken transfer while pretending to build product features.

- The project opens from the documented folder and the expected files are present with normal names.
- The host can start or the package launcher can start without missing Node, missing modules, blocked scripts, or stale disabled files.
- The phone-facing page can be reached through the documented local IP or tunnel path and shows the current UI rather than an old cached version.
- The worker knows whether they are editing source, testing a package, creating a GitHub issue, or preparing a release artifact.
- The worker knows which known issues are accepted for now, especially monitor switching, native capture, remote relay economics, and app-store constraints.

Files and state to protect

These are the project assets that should not be casually deleted, renamed, or overwritten during the transfer.

- `src/server.js`, `src/input-adapter.js`, `src/rtc-room.js`, `src/session-store.js`, `public/app.js`, `public/capture.js`, `public/host.js`, `public/sw.js`, and `packaging/local-wifi/*`.
- `docs/product_planning/*`, `README.md`, `MILESTONE_STATUS.md`, existing traceability docs, and checkpoint artifacts under `output/checkpoints`.
- `dist/OpenHostLink.zip` as the latest family-testable package, while treating it as a release artifact rather than source.
- Local secrets and runtime data under `data`, logs, PID files, and smoke-test extraction folders must be excluded from GitHub unless sanitized.

Verification before handoff is accepted

A handoff is not complete because files were copied. It is complete when the recipient can run and verify the system.

- The recipient can install dependencies or use the packaged zip without missing-file errors.
- The host console opens, shows the correct version, and exposes a phone URL/QR code.
- The test command listed in this manual passes or has an explicit, documented reason why a physical-device gate is still pending.
- The next Codex instance has a branch, issue, file ownership boundary, and acceptance gate before it starts editing.

Common mistakes to avoid

The transfer should avoid mistakes that already caused confusion during the prototype phase.

- Do not ship a package with multiple confusing launchers when one obvious launcher is expected.
- Do not let `.disabled-by-incident-response` file names become the canonical GitHub source state.
- Do not describe the monitor-switching system as fixed until it passes multi-monitor testing on another computer.
- Do not let a paid remote-access experiment break the free local Wi-Fi mode, because the free mode is the project's credibility base.

Acceptance evidence to attach

Every handoff page should produce evidence. If the page does not create evidence, it is probably only a note and not a real manual step.

- Attach command output or a short transcript for every command that matters.
- Attach a screenshot when the result is visual, such as host console, QR code, package folder, phone UI, capture window, settings sheet, or display selector.
- Attach file hashes for release zips so the sender and receiver can prove they are testing the same artifact.
- Attach a short failed-test note when something is known broken. A truthful failed-test note is more useful than a vague 'needs polish' sentence.
- Attach the next action owner so the handoff does not end in a pile of unassigned concerns.

Completion goals: a new worker can understand the page without chat history; the page gives concrete commands, files, or rules; and every named risk maps to the 25-page plan.

Page 8: Prompt Pack for New Instance

Page purpose: Paste this prompt into a fresh Codex instance after the repo exists: 'You are working on the Remote Controller project. Read docs/product_planning/Remote_Controller_Handoff_and_Transfer_Manual.docx and docs/product_planning/Remote_Controller_Product_Execution_Plan_25_Pages.docx first. Work only on issue and branch . Preserve local Wi-Fi mode, host approval, real-input safety, v76 touchpad relative behavior, and package reproducibility. Do not edit files outside your ownership without asking. Before final response, run the issue's tests and update docs if behavior changed.'

- Give the instance its chapter/page assignment.
- Give it the exact repo branch and file ownership.
- Give it one acceptance gate.
- Require a concise final report with evidence.

Exact operator actions

For 'Prompt Pack for New Instance', the operator should follow these steps rather than improvising from memory.

- Read the whole page once before touching files, Git, GitHub, package artifacts, or another computer.
- Capture current evidence first: folder path, branch or no-branch state, package version, latest zip path, and the commands that currently pass.
- Perform the named transfer or handoff action in one small batch, then verify immediately before moving to the next batch.
- Record any deviation in the handoff notes so the next Codex instance does not assume the ideal path happened.

Step-by-step handoff script

This is the minimum practical script for 'Prompt Pack for New Instance'. The point is to make the transfer repeatable for a new machine or a new Codex instance, not just understandable to the person who already lived through the prototype.

- Step 1: confirm the current workspace path and latest package path, then write both paths in the issue or handoff note before copying or editing anything.
- Step 2: run the listed smoke checks from the current machine so the sender knows whether the outgoing build is already healthy or already broken.
- Step 3: copy or publish only the approved source/package set. Exclude local runtime data, logs, PID files, extracted smoke folders, `node_modules`, and disabled scratch files.
- Step 4: on the receiving machine, launch from the documented command or the single packaged launcher and verify that the host page, phone URL, QR code, PIN, pairing, and real-input toggle appear as expected.
- Step 5: run the core automated tests from the receiving source checkout, or if using only the family package, run the package smoke and one phone pairing test.
- Step 6: record the result with machine name, Windows version, browser version, phone type, network type, and any missing permissions or firewall prompts.

What must be true before the next person starts

A new worker should not begin creative implementation until the handoff basics are proven. This prevents debugging a broken transfer while pretending to build product features.

- The project opens from the documented folder and the expected files are present with normal names.
- The host can start or the package launcher can start without missing Node, missing modules, blocked scripts, or stale disabled files.
- The phone-facing page can be reached through the documented local IP or tunnel path and shows the current UI rather than an old cached version.
- The worker knows whether they are editing source, testing a package, creating a GitHub issue, or preparing a release artifact.
- The worker knows which known issues are accepted for now, especially monitor switching, native capture, remote relay economics, and app-store constraints.

Files and state to protect

These are the project assets that should not be casually deleted, renamed, or overwritten during the transfer.

- `src/server.js`, `src/input-adapter.js`, `src/rtc-room.js`, `src/session-store.js`, `public/app.js`, `public/capture.js`, `public/host.js`, `public/sw.js`, and `packaging/local-wifi/*`.
- `docs/product_planning/*`, `README.md`, `MILESTONE_STATUS.md`, existing traceability docs, and checkpoint artifacts under `output/checkpoints`.
- `dist/OpenHostLink.zip` as the latest family-testable package, while treating it as a release artifact rather than source.
- Local secrets and runtime data under `data`, logs, PID files, and smoke-test extraction folders must be excluded from GitHub unless sanitized.

Verification before handoff is accepted

A handoff is not complete because files were copied. It is complete when the recipient can run and verify the system.

- The recipient can install dependencies or use the packaged zip without missing-file errors.
- The host console opens, shows the correct version, and exposes a phone URL/QR code.
- The test command listed in this manual passes or has an explicit, documented reason why a physical-device gate is still pending.
- The next Codex instance has a branch, issue, file ownership boundary, and acceptance gate before it starts editing.

Common mistakes to avoid

The transfer should avoid mistakes that already caused confusion during the prototype phase.

- Do not ship a package with multiple confusing launchers when one obvious launcher is expected.
- Do not let `.disabled-by-incident-response` file names become the canonical GitHub source state.
- Do not describe the monitor-switching system as fixed until it passes multi-monitor testing on another computer.
- Do not let a paid remote-access experiment break the free local Wi-Fi mode, because the free mode is the project's credibility base.

Acceptance evidence to attach

Every handoff page should produce evidence. If the page does not create evidence, it is probably only a note and not a real manual step.

- Attach command output or a short transcript for every command that matters.
- Attach a screenshot when the result is visual, such as host console, QR code, package folder, phone UI, capture window, settings sheet, or display selector.
- Attach file hashes for release zips so the sender and receiver can prove they are testing the same artifact.
- Attach a short failed-test note when something is known broken. A truthful failed-test note is more useful than a vague 'needs polish' sentence.
- Attach the next action owner so the handoff does not end in a pile of unassigned concerns.

Completion goals: a new worker can understand the page without chat history; the page gives concrete commands, files, or rules; and every named risk maps to the 25-page plan.

Research Sources Used

S1 - MDN getDisplayMedia:
https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getDisplayMedia
Browser screen capture prompts the user, cannot persist permission, and requires transient activation.

S2 - Parsec technology: https://parsec.app/technology
Low latency comes from native control, hardware encode/decode, zero-copy GPU pipeline, frame timing, and NAT traversal.

S3 - RustDesk GitHub: https://github.com/rustdesk/rustdesk
Open-source remote desktop can use vendor rendezvous/relay, self-hosted relay, or custom rendezvous/relay infrastructure.

S4 - Cloudflare Realtime TURN FAQ: https://developers.cloudflare.com/realtime/turn/faq/
Cloudflare TURN is priced at $0.05/GB with a 1,000 GB free tier and free STUN.

S5 - Twilio Network Traversal pricing: https://www.twilio.com/en-us/stun-turn/pricing
Twilio STUN is free and TURN is regionally priced, starting around $0.400/GB in US/EU regions.

S6 - Tailscale free plans: https://tailscale.com/docs/account/manage-plans/free-plans-discounts
Tailscale Personal permits 6 free users in one tailnet and has a GitHub community option for open-source projects.

S7 - Apple App Review Guidelines: https://developer.apple.com/app-store/review/guidelines/
Remote desktop clients need to be generic mirrors of user-owned host devices and follow store/privacy rules.

Source-to-Workstream Usage Rules

This appendix page exists so citations become actionable build rules. Future workers should use the links to decide what is technically possible, what must be measured, and what must be disclosed to users.

- â€¢ For browser capture work, use MDN as the constraint source. If a proposed fix depends on the browser silently choosing the whole screen, remembering permission forever, or launching capture without user activation, the worker must label that as browser-limited and route the finished paid-product plan toward native host capture.
- â€¢ For low-latency work, use Parsec as the performance comparison point. The project does not need to clone Parsec, but any claim about a zero-delay system should identify where the current pipeline still differs: native capture, hardware encode, hardware decode, frame pacing, UDP transport quality, jitter buffer behavior, and GPU copy count.
- â€¢ For open-source/self-host strategy, use RustDesk as evidence that rendezvous and relay architecture can be made user-controlled. This does not mean copying RustDesk; it means the architecture should keep signaling, relay, and host trust separable so the free product can remain credible and the paid product can scale.
- â€¢ For TURN economics, update Cloudflare and Twilio pricing before any public subscription promise. The worker should calculate relay GB per hour at multiple quality levels and should record the percentage of sessions expected to be direct P2P versus relayed.
- â€¢ For Tailscale, keep the guidance narrow: it is a useful free workaround for technical users and private family setups, not a replacement for the product's paid remote-access onboarding. Do not make a normal user install a VPN just to understand the app.
- â€¢ For Apple review, treat the app as a remote desktop client for user-owned host computers. Avoid language or features that make it look like hosted app streaming, a store inside a store, hidden surveillance, or unauthorized device control.

Verification Rule for Future Research

Pricing, browser behavior, app-store rules, and mobile OS capabilities can change. Before shipping, submitting, pricing, or marketing the app, the responsible worker must reopen the relevant primary source, record the date checked, and update the issue.

- â€¢ Do not rely on this document alone for final legal, pricing, or platform decisions.
- â€¢ Do not use second-hand blog posts as the only authority when a primary source is available.
- â€¢ If a source changes, update the product plan and the acceptance gate that depended on it.

Issue Conversion Rules

The source appendix is complete only when it becomes work. The first GitHub migration should convert these source constraints into concrete issues so nobody has to rediscover the same limits during launch pressure.

- â€¢ Create one GitHub issue for browser-capture limitations and link it to the native host capture spike. The issue should explicitly say which behaviors are impossible or unreliable in a normal browser and which behaviors can remain in the free local fallback.
- â€¢ Create one GitHub issue for low-latency measurement. It should require a test video, moving cursor path, input RTT log, WebRTC stats export, and a before/after comparison for each capture or rendering optimization.
- â€¢ Create one GitHub issue for relay economics. It should include direct-vs-relay percentage, bandwidth per hour, expected free-tier usage, paid-tier fair use, provider pricing checked date, and the point where self-hosted relay becomes cheaper.
- â€¢ Create one GitHub issue for mobile store compliance. It should collect iOS and Android privacy labels, account deletion requirements, host-device ownership language, subscription rules, support/demo mode, and review notes.
- â€¢ Create one GitHub issue for free local/open-source positioning. It should decide which parts can be public, which assets stay private, how family-test packages are built, and how local mode stays useful even if paid services are down.